

1. Introduction

In this project I aim to determine an equation for the distribution of meteorite impacts in a given cluster. Expectedly this data would be random as meteorites do not have a given choice of where to strike the earth except perhaps near the equator as it lays in the trajectory of the asteroid belt. That said, it is still an interesting analysis as it may lead to some remarkable discoveries about meteorite impact locations and distributions.

2. Data

The data chosen is meteorite landings from The Meteoritical Society, published by NASA, and retrieved from data.gov (<https://catalog.data.gov/dataset/meteorite-landings>). This dataset includes all known meteorite landings over the last 200 years. It contains the name, id, nametype, validity, date of impact, mass, geolocation, etc. for all landings.

3. Filtering Data

The data includes plenty of irrelevant information. For example, the nametype, recclass, date of impact, and mass are not important to my analysis of impact distributions. Additionally, geolocation is included twice, once a latitude and longitude and again as the same but in a tuple. Finally, it includes if the meteorite was observed falling or if it were just found. Since most meteorites (all but around 500) were found, it makes little sense to filter the data based on that metric.

All irrelevant columns were removed. The tuple variant of geolocation and the fall data were also removed.

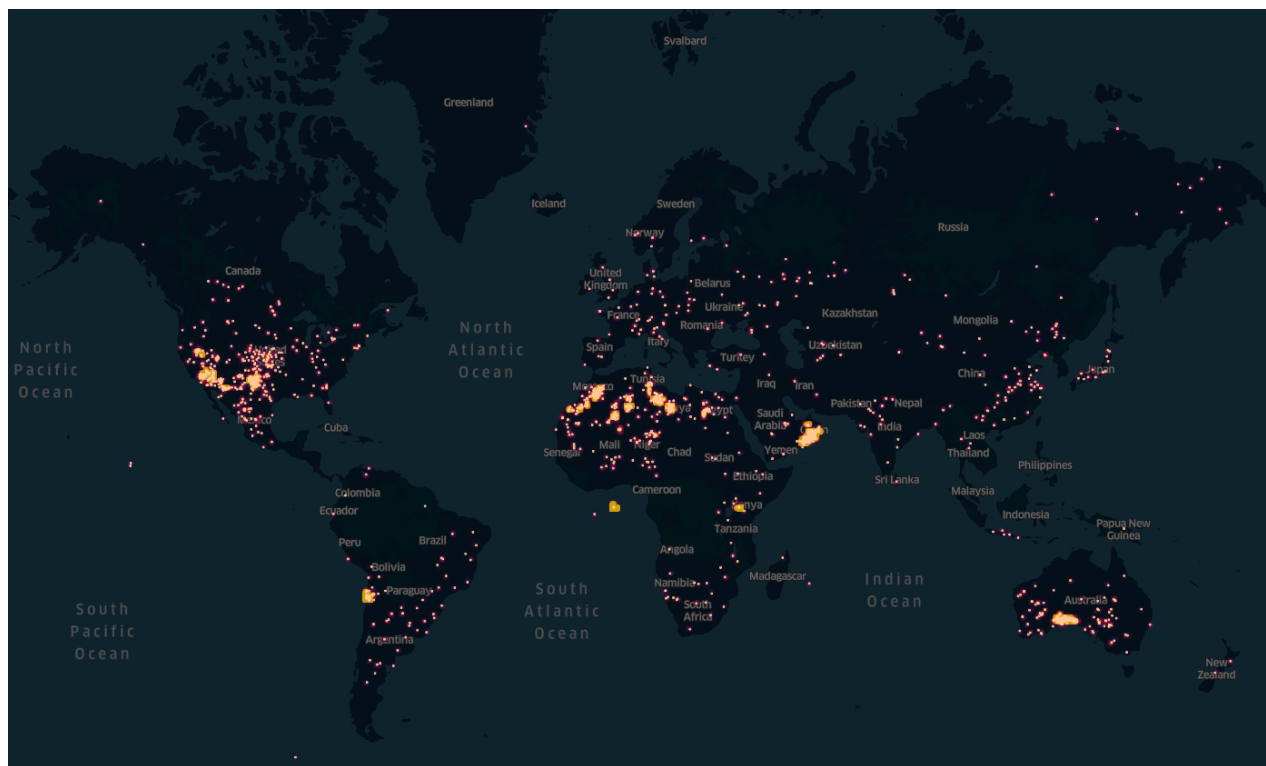
Next, some of the data is tremendously old (1800s!). All impacts before 1970 are eliminated as I do not trust data sourced before then. Next some data did not contain valid latitude and longitude coordinates. Such data was also eliminated. The original data contained ~45,000 rows, and the filtered data contains ~35,000 rows. This leaves enough data for reliable analysis and also eliminates all questionable data.

All filtering was handled by pandas.

4. Fitting Data Expectations

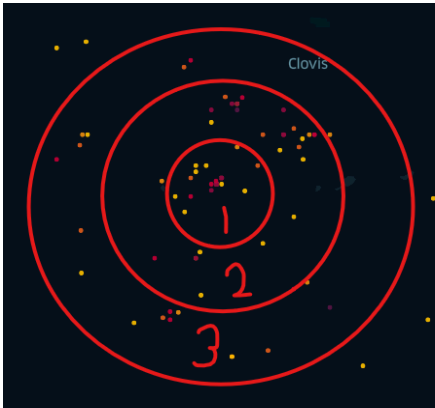
A cluster is defined as a 500 km radius from impact. I originally chose 1000 km but that became troublesome for densely cluster-filled regions where a 1000 km radius may include multiple clusters. The data was originally split into histograms of 10 km chunks but after curve fitting and error analysis, I switched to using 1 km chunks as it minimized the error significantly.

I parsed the data with geopy and plotted the filtered input data using kepler.gl. I created a point-plot and a heatmap. Combining the two gave the following result (polar regions removed for brevity).



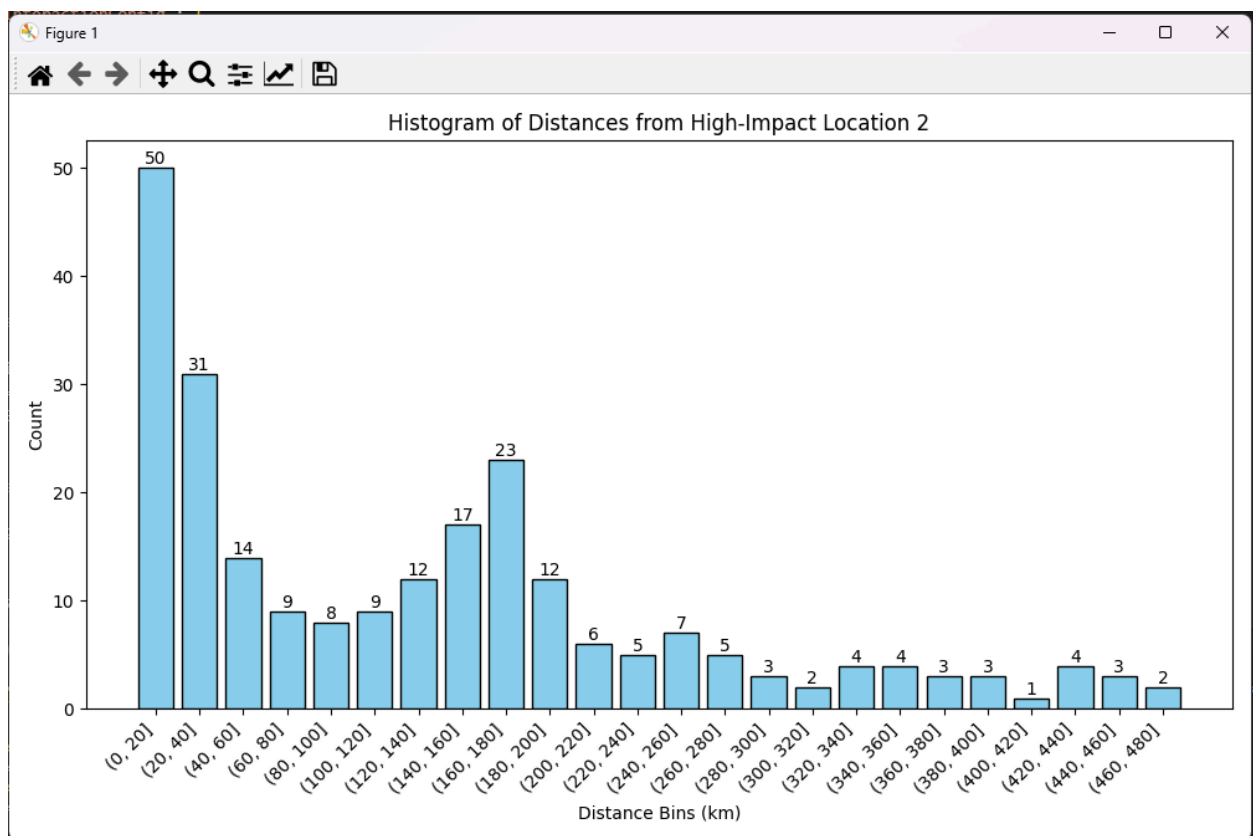
At this point I manually found large clusters and noted them for further analysis (to guess initial parameters for curve fitting and to serve as center locations for analysis).

After inspection, I suspected either a gaussian, decay, or a linear curve would fit best. Gaussian makes sense since even centering the analysis at the most dense location, it is possible, radially, that further out contains more meteorites.

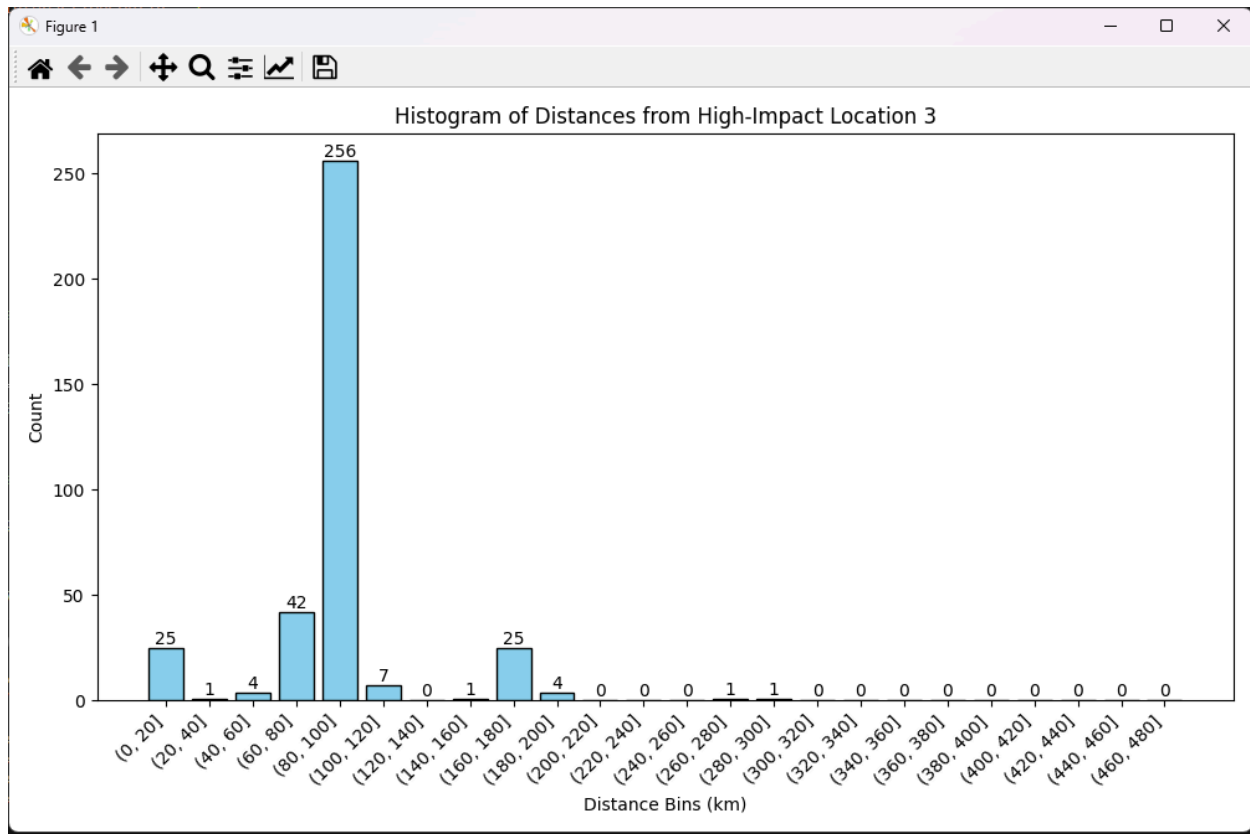


For example, it can be seen that even if we center our analysis in circle one (the most densely packed one) it is still possible that ring 2 contains more meteorites due to its area of coverage being larger, and declination only begins in ring 3. Such a scenario will be well reflected in a gaussian model. This was not the case.

Decay is a rather obvious choice given the data is centered in the most dense regions. Linear is a backup (basically a slower exponential decay). Linear was unneeded.



This is an example of a possible exponential decay function (or maybe polynomial?),

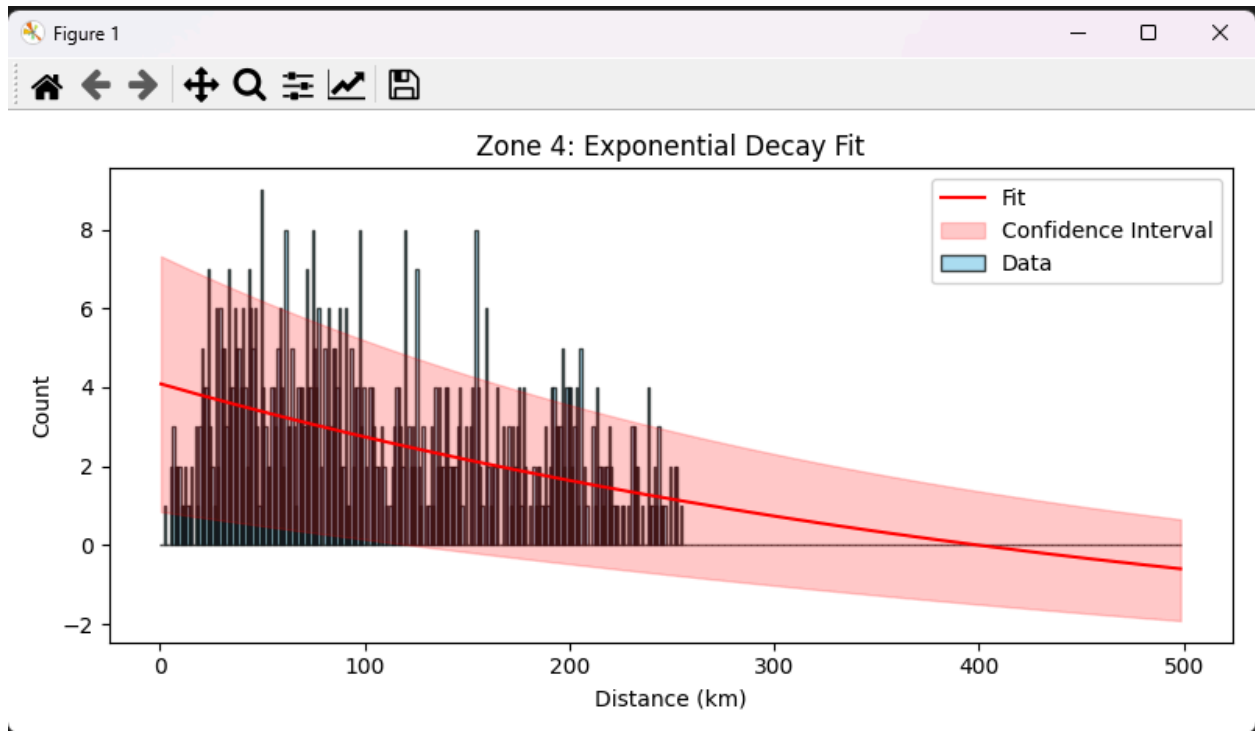


This data is closer to a gaussian distribution. Although it looks like the center is poorly chosen, that is not the case. This describes the situation earlier depicted by the rings.

5. Fitting Data Plan and Testing

To fit the data, I first split it into a histogram with 10 km (though I later switched to 1 km) chunks. Then I used the histogram bins as the x-coordinate and the number of meteorites in each bin as a y-coordinate. Treating these as a scatter plot, I used scipy to curve fit and generate an equation for the data.

To see if this would work, I generated some test data using an exponential function as I suspected that to be the most likely distribution for the data. The data centers itself at a random point on the globe and plots an exponentially decreasing distribution from that point. This is done for multiple locations and that data is then stored along with the locations. The fitted curve overlaid on the original data looks as such.



And the resulting equation is:

$$P(x) = ae^{-bx} + c$$

$$P(x) = -16.106\exp(-0.0013 * x) + 11.50$$

$$a: 16.10 \pm 13.80$$

$$b: 0.0013 \pm 0.0007$$

$$c: -11.50 \pm 13.96$$

6. Fitting Data Reality

Running scipy's curve_fit on the data with 10 km bins and a 500 km cluster radius r resulted in the following equation of the exponential form:

$$P(x) = ae^{-bx} + c$$

$$P(x) = -21347.56\exp(-0.0721 * x) + 22333.04$$

$$a: -21347.56 \pm 154831691.18$$

$$b: 0.0721 \pm 441.2994$$

$$c: 22333.04 \pm 117031746.38$$

7. Error analysis and Refitting

These errors are obviously massive! To reduce these errors I played around with the bin size and the cluster radius. After some manipulation, the lowest error was achieved with a bin size of 1 km and a cluster radius of 500 km. Increasing the radius toward 1000 km and down toward 300 km both increased the error.

The equation with 1 km bin size and 500 km cluster size is:

$$P(x) = ae^{-bx} + c$$

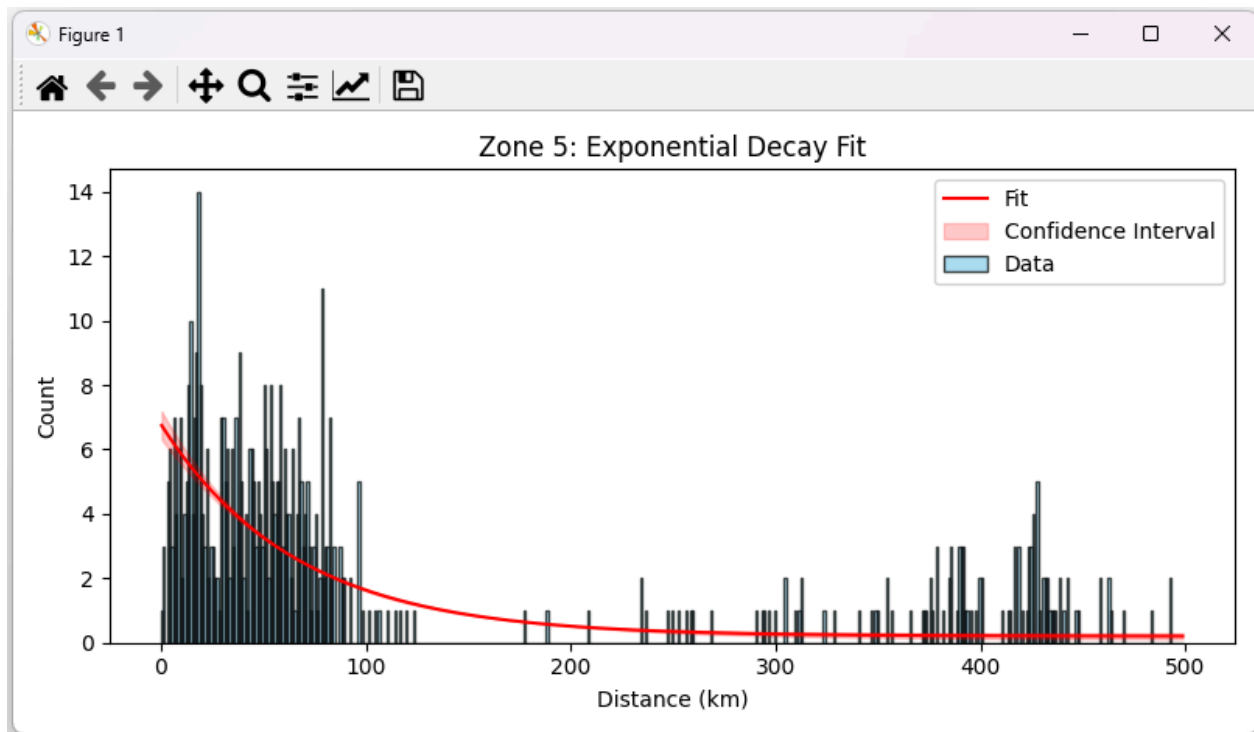
$$P(x) = -10.76\exp(-0.0731 * x) + 2.60$$

$$a: 10.76 \pm 4.32$$

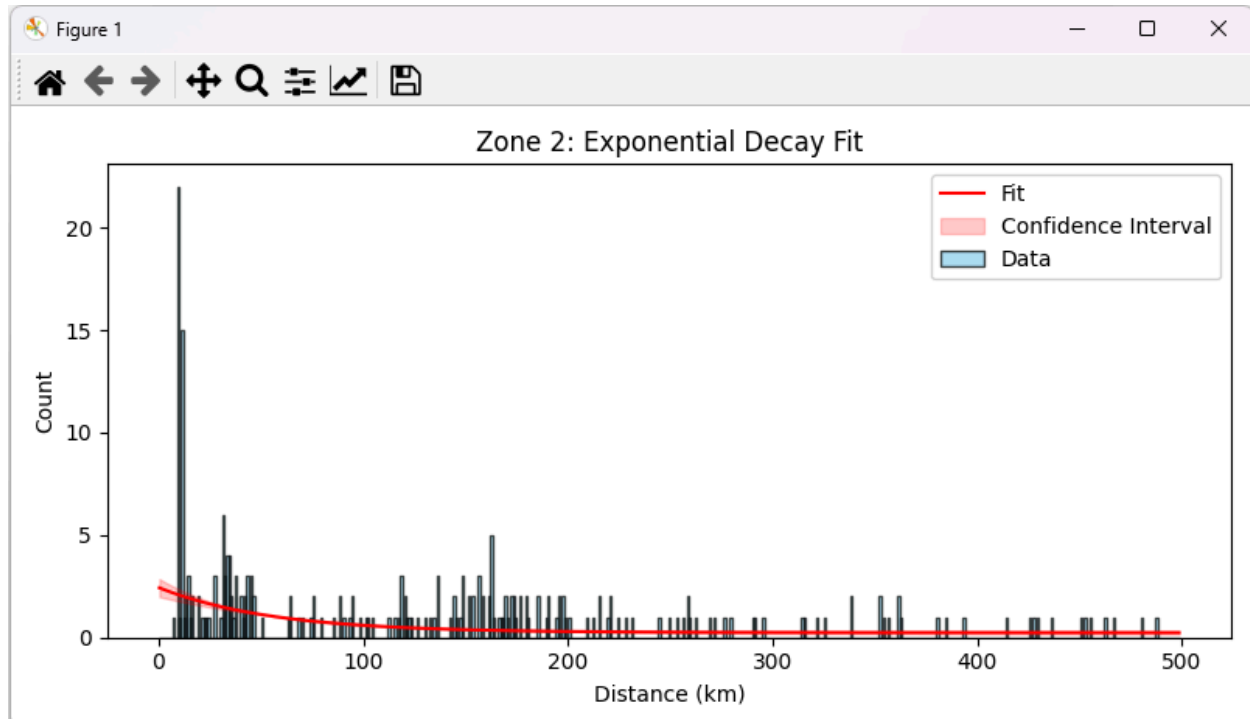
$$b: 0.0731 \pm 0.0159$$

$$c: -2.60 \pm 4.10$$

Here are some examples of the fitted curved overlaid on the original data, including error.



Here we can almost see another cluster starting to develop. This may be resolved by an algorithm built to detect clusters and change the cluster radius dynamically.

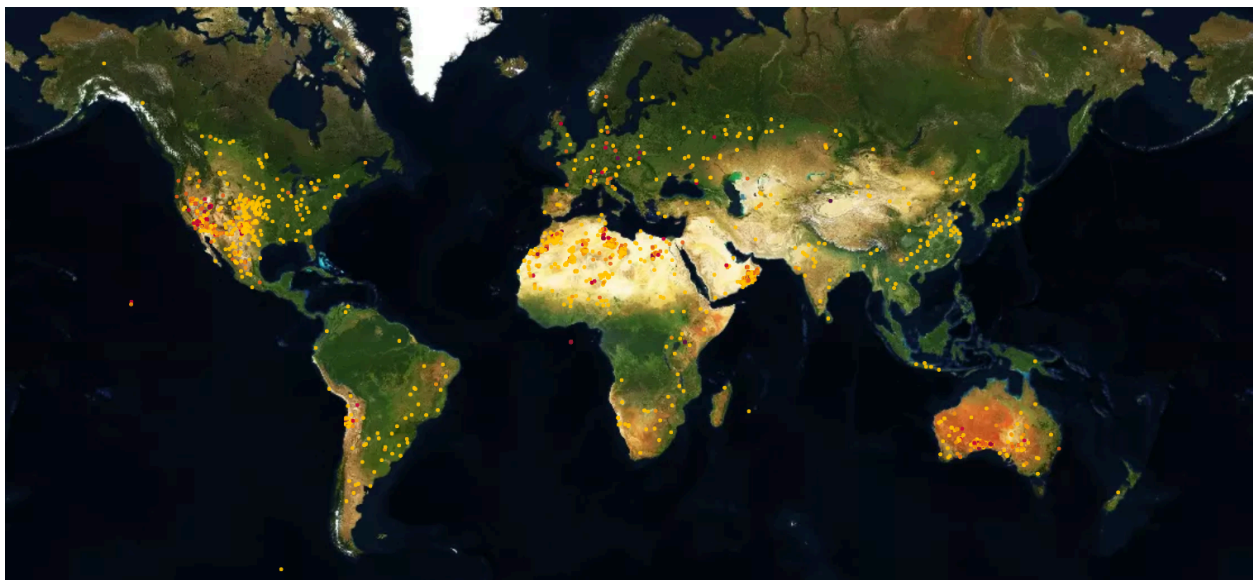


This is an example of a nice fit to the data.

8. Conclusion

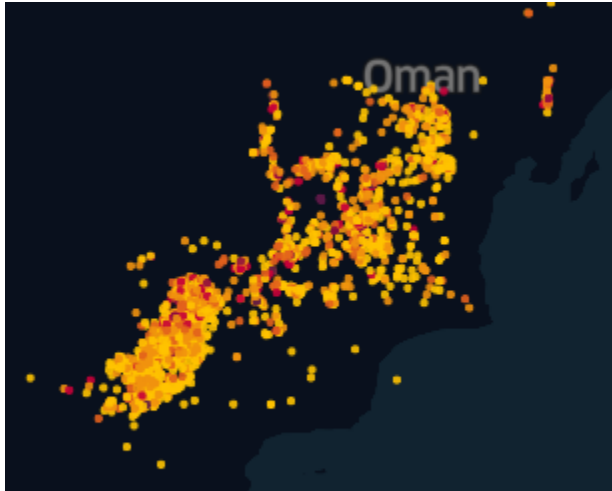
While an equation was successfully determined and seems to match the data fairly well, it is hard to claim it to be a good model. Partially because the error is fairly high. Additionally, the equation only represents the decay trend, not the actual locations of the meteorite impacts. One cannot use this equation to determine where to look, only roughly gain a probability function. Nothing more.

Additionally, looking at the point-plot on a satellite image shows another story.



Most of the points are simply in desert areas. This is primarily because impacts are simply easier to detect and study there. Moreover, history is preserved better in deserts than, for example, the tropics. Notably, the Amazon, middle africa, the Philippines, etc. do not have many impacts.

Finally, areas like Oman are meteorite heaven.



There is no observable decay of any kind there. This is quite difficult to model and does not fit the other data well (the other does display some type of decay). That said, the final equation is the following:

$$P(x) = ae^{-bx} + c$$

$$P(x) = -10.76\exp(-0.0731 * x) + 2.60$$

$$a: 10.76 \pm 4.32$$

$$b: 0.0731 \pm 0.0159$$

$$c: -2.60 \pm 4.10$$

With x being the center of a cluster and the equation predicting the trend of meteorite impacts in a 500 km radius.

Libraries used: pandas, numpy, matplotlib, keplergl, os, scipy, and geopy